

■ Movie Review Sentiment Analysis

IMDB Dataset — Machine Learning Classification Report

Dataset	IMDB Movie Reviews
Total Records	50,000 Reviews
Task	Binary Sentiment Classification
Classes	Positive / Negative
Tool	Python — Scikit-learn, NLTK, XGBoost

1. Problem Statement

With the rapid rise of user-generated content on platforms such as IMDb, Rotten Tomatoes, and social media, it has become essential for companies and consumers to understand the general sentiment expressed in movie reviews. This project aims to build a machine learning pipeline that automatically classifies movie reviews as **Positive** or **Negative** based on their textual content.

The IMDB dataset containing 50,000 labeled movie reviews is used for training and evaluating multiple classification algorithms, comparing their performance to identify the best-suited model for this task.

2. Data Collection & Initial Exploration

2.1 Dataset Overview

The dataset used in this project is the **IMDB Movie Review Dataset**, a widely-used benchmark for sentiment analysis tasks. It consists of 50,000 movie reviews collected from the Internet Movie Database (IMDB), each labeled as either positive or negative.

Property	Detail
Source	IMDB Movie Reviews Dataset
Total Rows	50,000
Total Columns	2 (review, sentiment)
Classes	Positive, Negative
Class Distribution	25,000 Positive 25,000 Negative (50-50)
Null Values	0 (No missing data)
Duplicate Rows	418 (removed before processing)
Data Types	review: object sentiment: object

2.2 Key Observations

- The dataset is **perfectly balanced** — 50% positive and 50% negative reviews. This eliminates the need for oversampling or undersampling techniques.
- There are **no null values**, so no imputation was required.
- **418 duplicate rows** were found and removed using `df.drop_duplicates()`.
- Reviews contain **HTML tags** (e.g., `<br />`), **URLs**, **punctuation**, and **slang words** that needed to be cleaned before modelling.
- The `df.describe()` output showed the most frequent sentiment label is **positive** with a frequency of 25,000.

3. Data Preprocessing

Raw text data cannot be directly fed into machine learning models. A thorough preprocessing pipeline was applied to clean and normalize the text before feature extraction.

3.1 Lowercasing

All review text was converted to lowercase using *str.lower()* to ensure uniformity. This prevents the model from treating 'Good' and 'good' as different words.

3.2 Removing HTML Tags

IMDB reviews frequently contain HTML tags such as `<br />`. A regular expression pattern `r'<.*?>'` was used to strip all HTML tags from the reviews.

3.3 Removing URLs

URLs were removed using the pattern `r'https?://[A-Z|a-z|www|.]+'` to eliminate irrelevant web addresses that carry no sentiment information.

3.4 Removing Punctuation

All punctuation characters defined in Python's *string.punctuation* were removed character by character from each review to reduce noise.

3.5 Chat/Slang Word Conversion

A custom dictionary of 80+ internet slang terms (e.g., LOL → Laughing Out Loud, BRB → Be Right Back) was used to convert abbreviations into their full forms, preserving the semantic meaning.

3.6 Emoji Handling

The *emoji* library was used to convert emojis into their textual descriptions using *emoji.demojize()*, ensuring emotional cues from emojis are not lost.

3.7 Removing Stopwords

Common English stopwords were removed using NLTK's *stopword* list. However, important negation words such as 'not', 'no', 'never', 'don't', 'couldn't' were explicitly kept as they carry strong sentiment signals (e.g., 'not good' is very different from 'good').

3.8 Removing Extra Spaces

Extra whitespace introduced during preprocessing was cleaned using `" ".join(text.split())`.

3.9 Tokenization

Each review was tokenized into individual words using NLTK's *word_tokenize()* function, splitting the text into a list of tokens for further processing.

3.10 Stemming

Porter Stemmer from NLTK was applied to reduce words to their root form (e.g., 'running' → 'run', 'wonderful' → 'wonder'). This reduces the vocabulary size and helps the model generalize better.

4. Feature Engineering

4.1 Label Encoding

The target column *sentiment* was encoded using **LabelEncoder** from scikit-learn. The string labels were converted to binary integers: **negative = 0, positive = 1**.

4.2 Bag of Words (CountVectorizer)

CountVectorizer was applied with *max_features=5000* to create a document-term matrix representing word frequencies. Each review becomes a vector of word counts across the 5,000 most frequent tokens.

4.3 TF-IDF Vectorization (Used for Modelling)

TF-IDF (Term Frequency-Inverse Document Frequency) was used as the primary feature extraction method for model training. It weighs words by how frequently they appear in a document relative to how common they are across all documents — reducing the impact of very common but uninformative words.

Parameter	Value	Reason
max_features	10,000	Captures a wider vocabulary
ngram_range	(1, 2)	Includes bigrams (e.g., "not good")
min_df	2	Ignores very rare terms
max_df	0.95	Ignores overly common terms
sublinear_tf	True	Applies log normalization to term frequency

4.4 Train-Test Split

The dataset was split into **80% training** and **20% testing** sets using *train_test_split* with *random_state=42* for reproducibility. This resulted in approximately **39,000 training samples** and **9,917 test samples**.

5. Model Selection & Fitting

Six machine learning models were trained and evaluated on the TF-IDF feature matrix. Each model was selected to represent a different algorithmic approach to classification.

Logistic Regression

Parameters: C=4, max_iter=2000, solver='liblinear'

A linear model that estimates the probability of a class using a logistic function. Highly effective for high-dimensional text data.

Naive Bayes (MultinomialNB)

Parameters: alpha=0.3

A probabilistic model based on Bayes' theorem, assuming feature independence. Known to work very well with text classification tasks.

Decision Tree

Parameters: criterion='gini', max_depth=100, min_samples_split=20

A tree-based model that splits data based on feature thresholds. Simple to interpret but prone to overfitting on high-dimensional data.

Random Forest

Parameters: *n_estimators=200, max_depth=50, n_jobs=-1*

An ensemble of decision trees trained on random subsets of data. More robust than a single decision tree and reduces overfitting.

SVM (LinearSVC)

Parameters: *C=1.5, max_iter=5000*

Support Vector Machine finds the optimal hyperplane to separate classes. LinearSVC is efficient and highly effective for text classification.

XGBoost

Parameters: *n_estimators=200, learning_rate=0.1, max_depth=6*

A gradient boosting algorithm that builds trees sequentially, each correcting the errors of the previous. Powerful on structured data.

6. Model Evaluation & Results

6.1 Overall Accuracy Comparison

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	89.50%	0.90	0.89	0.90
Naive Bayes	86.51%	0.87	0.87	0.87
Decision Tree	72.66%	0.73	0.73	0.73
Random Forest	84.51%	0.85	0.85	0.85
SVM (LinearSVC)	88.36%	0.88	0.88	0.88
XGBoost	84.66%	0.85	0.85	0.85

6.2 Logistic Regression — Accuracy: 89.50%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.90	0.88	0.89	4,939
Positive (1)	0.89	0.90	0.90	4,978
Macro Avg	0.90	0.89	0.90	9,917
Weighted Avg	0.90	0.90	0.90	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	4,371	568
Actual Pos	473	4,505

Logistic Regression achieved the highest accuracy of 89.50%. The model performed consistently across both classes with balanced precision and recall. Only 568 negative reviews were misclassified as positive

and 473 positive reviews as negative, making it the best overall performer.

6.3 Naive Bayes — Accuracy: 86.51%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.88	0.85	0.86	4,939
Positive (1)	0.85	0.88	0.87	4,978
Macro Avg	0.87	0.87	0.87	9,917
Weighted Avg	0.87	0.87	0.87	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	4,185	754
Actual Pos	584	4,394

Naive Bayes performed well with 86.51% accuracy. It is a fast and simple probabilistic model. Despite the independence assumption being rarely true in real text, it still delivers solid results. It misclassified 754 negative reviews as positive, slightly more than Logistic Regression.

6.4 Decision Tree — Accuracy: 72.66%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.72	0.74	0.73	4,939
Positive (1)	0.73	0.72	0.73	4,978
Macro Avg	0.73	0.73	0.73	9,917
Weighted Avg	0.73	0.73	0.73	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	3,632	1,307
Actual Pos	1,404	3,574

Decision Tree had the lowest accuracy at 72.66%. While it is interpretable, it struggles with high-dimensional TF-IDF features, leading to overfitting. The high number of misclassifications (1,307 + 1,404) compared to other models confirms it is not well-suited for this task.

6.5 Random Forest — Accuracy: 84.51%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.86	0.82	0.84	4,939
Positive (1)	0.83	0.87	0.85	4,978

Macro Avg	0.85	0.85	0.85	9,917
Weighted Avg	0.85	0.85	0.85	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	4,058	881
Actual Pos	655	4,323

Random Forest improved over a single Decision Tree by 12%, reaching 84.51%. The ensemble approach reduces overfitting. However, it is computationally expensive and still falls behind Logistic Regression and SVM for this text classification problem.

6.6 SVM (LinearSVC) — Accuracy: 88.36%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.89	0.88	0.88	4,939
Positive (1)	0.88	0.89	0.88	4,978
Macro Avg	0.88	0.88	0.88	9,917
Weighted Avg	0.88	0.88	0.88	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	4,326	613
Actual Pos	541	4,437

SVM (LinearSVC) achieved 88.36% accuracy, second only to Logistic Regression. SVM is well-known for performing excellently in high-dimensional sparse feature spaces like TF-IDF. The model showed very balanced precision and recall across both classes.

6.7 XGBoost — Accuracy: 84.66%

Classification Report:

Class	Precision	Recall	F1-Score	Support
Negative (0)	0.87	0.82	0.84	4,939
Positive (1)	0.83	0.87	0.85	4,978
Macro Avg	0.85	0.85	0.85	9,917
Weighted Avg	0.85	0.85	0.85	9,917

Confusion Matrix:

	Predicted Neg	Predicted Pos
Actual Neg	4,041	898
Actual Pos	623	4,355

XGBoost achieved 84.66% accuracy, similar to Random Forest. While XGBoost is a powerful gradient boosting algorithm, it is generally more effective on structured/tabular data. For sparse TF-IDF text features, linear models like Logistic Regression and SVM outperform it.

7. Conclusion

This project successfully built an end-to-end movie review sentiment analysis pipeline using the IMDB dataset. Six machine learning models were trained and evaluated after thorough text preprocessing and TF-IDF feature extraction.

Key Findings:

- **Logistic Regression** achieved the highest accuracy of **89.50%** and is the recommended model for this task.
- **SVM (LinearSVC)** came in second with **88.36%**, also highly suitable for text classification.
- **Naive Bayes** delivered competitive performance at **86.51%** with very fast training time.
- **Random Forest** and **XGBoost** achieved around **84-85%**, showing ensemble methods work but are outperformed by linear models on sparse text features.
- **Decision Tree** had the lowest accuracy at **72.66%**, struggling with the high-dimensional TF-IDF feature space.
- The **perfectly balanced dataset** (50/50) ensured fair evaluation across all models without class bias.
- Keeping **negation words** during stopwords removal was a crucial preprocessing decision to preserve sentiment signals.
- Using **bigrams (ngram_range=(1,2))** in TF-IDF helped capture phrases like 'not good' or 'very bad'.

Recommended Model: Logistic Regression

Logistic Regression is recommended as the final model due to its highest accuracy (89.50%), balanced precision-recall across both classes, fast training and inference time, and excellent performance on high-dimensional sparse TF-IDF features. It is also highly interpretable — the model coefficients directly indicate which words are most predictive of positive or negative sentiment.